

KAFRIADA NET — Frontend Design Plan

Oct 6, 2026 · @Idris

Summary

We rebuild the KAFRIADA frontend from zero as "**Grassroots Matchday**": a mobile-first football product that feels like a player card, a matchday programme and a referee's whistle, while still working on a cheap Android phone running Opera Mini with no JavaScript.

The current frontend is not badly engineered. Its data layer, security headers and no-JS forms are careful work. What fails is the design: it was styled as a passport (guilloche, perforations, serif type), it says nothing about football, it clips sideways on a 414px phone, and parts of the text vanish in dark mode. The backend repo's `web/` folder is also far ahead of the `kafriadadev_frontend` repo (22 screens built against about 10), so the two copies have already drifted.

The plan in one line: keep the plumbing, burn the paint. The agent wipes every style, layout and component, rebuilds a new design system around the logo's green, red and black, and re-skins all 46 wireframed screens in nine phases, launch screens first.

The one hard constraint behind every choice: the project's own rule 7 says every launch screen works with JavaScript off, inside a 150KB first load on 2G. So the animation you want is built in three layers. CSS motion works everywhere, richer JavaScript motion arrives only where the phone can take it, and nothing ever depends on it.

Five decisions only you can make

1. **Brand name.** The logo says KAFRIADA NET, Sport Network Platform. The code, card and docs say KAFRIADA and KAFRIADA CORE, issued by the Federation of Nigerian Sports. Which name goes on the site and the printed card?
2. **Football only, or multi-sport?** Registration offers ten sports today. This plan makes football the visual language either way. If other sports stay, they keep a dropdown but lose the pitch-position picker.
3. **Which repo is real?** The backend's own TODO asks this. Recommendation: build the new frontend in `kafriadadev_frontend`, port logic from `kafriadadev_backend/web` (the newer copy), then delete `web/` once the new one reaches parity.
4. **Hausa at launch?** Recommendation: yes, English and Hausa from day one, because the pilot is Jigawa. Who reviews the Hausa copy?

5. **Installable app (PWA)?** It is in the Pilot Build Spec and never built. Recommendation: yes, in the final phase.

One knock-on to know now: the printed card's PNG and PDF are drawn by the backend (`api/.../identity/card.py`, with its own fonts). A new card design means that renderer must be updated too, or the download will not match the screen.

Audit of what exists

Two copies of the frontend exist, and neither speaks football.

Copy	Screens built	Last real work	Status
<code>kafriadadev_backend/web/</code>	22 of 46 (public, auth, athlete, verification, pay, clubs, coordinator, admin)	Current, alongside the API	The real one today
<code>kafriadadev_frontend</code>	About 10 (public, register, card, sign-in, forgot, me)	12 September	Stale snapshot, already drifted

The design problems, from the code and the two screenshots in `docs/screens/` :

- **Wrong concept.** The stylesheet's stated direction is "security document": passports, banknotes, guilloche, perforated edges, an MRZ strip, a serif display face. A young footballer sees a government form, not a sports platform.
- **Breaks on phones.** The public profile screenshot at 414px clips the name, the KUID and the facts column off the right edge. That is the highest-traffic page in the system.
- **Text disappears in dark mode.** The athlete's name and the KUID render almost white on cream, because the card is pinned light while some ink follows the theme.
- **Styling scattered inline.** Dozens of `style={{...}}` objects in every page, so there is no system to change in one place.
- **No icons, no imagery, no motion.** The stylesheet bans animation outright, citing the 2G budget.
- **Errors are generic boxes.** A red-bordered notice with a sentence. No icon, no action button, and the reference code the wireframes require on every error is missing from most screens.
- **No navigation.** One link (Sign in) in the header. Athletes, coordinators and admins have no persistent way around.

What is good, and why the plan keeps it: every form is a plain POST that works without JavaScript, the browser never talks to the API directly, sessions live in httpOnly cookies, and the copy is careful (for example, the profile says "Issued by KAFRIADA", never "Verified athlete", because a photocopied QR keeps a valid signature).

The wireframes PDF (`docs/KAFRIADA-CORE-Wireframes.pdf`) fixes what each of the 46 screens does and says nothing about how it looks. It stays the source of truth for content and states. This plan supplies the look.

The nuke rule and what survives

Before writing any new design, the agent deletes every visual decision in the old frontend: all CSS, every inline style, every class name, every layout, every component's markup, the fonts and the colour values. It starts from a fresh Next.js app and copies in only the files on the list below, each recorded in a `KEPT.md` at the repo root with one line on why. Anything not on this list that the agent wants to reuse must be added to `KEPT.md` and raised with you before it is used.

Kept: plumbing, not paint. None of these files decide how anything looks.

Kept	From	Why it survives
<code>lib/api.ts</code>	backend <code>web/</code>	The only door to the API: timeouts, error parsing, field-level errors. Rewriting it adds risk and no design value.
<code>lib/session.ts</code> , <code>lib/money.ts</code>	backend <code>web/</code>	httpOnly session and pending cookies; integer-kobo money formatting (project rule 1).
Every <code>actions.ts</code> (server actions)	backend <code>web/</code>	The form logic that makes no-JS submission work. The new pages call the same actions.
Route handlers: <code>qr/</code> , <code>card.pdf</code> , <code>card.png</code> , <code>photo/</code> , <code>review-media/</code> , club document route	backend <code>web/</code>	They proxy files from the API. No markup in them.
<code>next.config.ts</code> security headers	backend <code>web/</code>	Includes a hard-won fix: <code>Referrer-Policy: same-origin</code> , without which no-JS registration returns a 500.

Kept	From	Why it survives
<code>scripts/check-render.mjs</code>	backend web/	The test that proves pages work with JavaScript off. Extended, not replaced.
<code>Dockerfile</code>	backend web/	Deployment, unchanged.

Kept: words and rules, rewritten visually. You should know these carry over because they are product rules, not taste:

- The load-bearing copy: "Issued by KAFRIADA" on the profile, the line telling a scout to check the face, the privacy consent sentence, the statement that money can never be withdrawn, never shaming an unverified athlete.
- Every error screen shows a short reference code (a wireframe convention).
- The printed card does not invert in dark mode, so a photo of it always looks the same. This is the one old design idea the plan keeps, on purpose.
- The ID uses a font where 0 and O, 1 and l can never be confused. The font changes; the requirement stays.

Not kept, rebuilt fresh: `globals.css`, `layout.tsx` markup, `SubmitButton`, `Flash`, `ResendCountdown`, `AdminNav`, every page's JSX, Atkinson Hyperlegible, Instrument Serif and JetBrains Mono, the green-and-gold palette, the seal, the perforations and the MRZ strip.

Who uses this, and what they need to feel

The athlete wants to be **seen**; everyone else wants to **trust what they see, fast**. Every screen serves one of those two feelings.

The athlete (most users, by far)

A young man, 18 or older during the pilot, playing grassroots football somewhere in Jigawa's 27 LGAs. Hausa first, English second. A Tecno, Itel or Infinix phone, often shared, data bought in small bundles, Opera Mini common, screen read outdoors in sunlight. He heard about KAFRIADA from a poster, a coach or a WhatsApp forward.

- **What drives him: the dream of being discovered.** The ID matters because a scout or club might look at it. The design should make him feel like a player with a profile, not a citizen filling a form. The player card is the emotional centre of the product.

- **Pride and sharing.** The CEO plan calls the shareable profile "our marketing engine": athletes post it on WhatsApp, on jerseys, at trials. So the card must look good in a WhatsApp preview and in a screenshot, and sharing must be one tap.
- **Suspicion of money.** Many people have met "pay first" scams. Free must be unmistakable and come first, the ₦2,500 only after the ID is in his hand, and the cash route through his LGA coordinator must be as visible as the card route.
- **Low reading load.** Short sentences, one action per screen, icons beside every label, numbers big. Never rely on reading a paragraph to know what to do.
- **Fear of losing it.** A failed SMS must never look like a lost registration. Reassure at every step: "Your ID is already yours."

The LGA and state coordinator

A field officer in a hall with 200 people waiting, on a phone or cheap tablet shared with colleagues, sometimes handling cash. Needs speed and certainty: big targets, the queue count and cash total always visible, one decision per screen, unmistakable confirmation after every approve or payment. Errors here cost money and trust, so they must be loud and specific.

The scout, club official and pitch-side stranger

Scans a QR on a printed card with one bar of signal and wants one answer in two seconds: is this real, and is this the person in front of me? The public profile must load with zero JavaScript and put "Issued by KAFRIADA" and the photo above everything. Club admins also manage rosters: they want their squad to look like a squad.

The administrator

At a desk, on a laptop, looking at ledgers, audit logs and user roles. Wants density, tables, filters and calm. Football styling here is light: the brand colours and icons, not the matchday drama.

What this means for design

Principle	Rule the agent follows
Football is the language	Pitch lines, player cards, positions on a pitch, referee cards and whistles carry meaning, not decoration.
One screen, one job	One primary button per screen, always in the thumb zone on mobile.

Principle	Rule the agent follows
Icon plus word, always	No icon without a label, no colour without an icon or word (colour-blind safe, low-literacy safe).
Free first, money last	No price above the free deliverable on any athlete screen.
Reassure constantly	Every wait, failure and payment says what is already safe.
Speak Hausa	All copy in an i18n file from day one; Hausa is a full language, not a toggle afterthought.
Respect the data bundle	Images optional, no autoplay, honour the <code>Save-Data</code> header.

Design direction: Grassroots Matchday

The site should feel like the afternoon of a local match: white chalk lines on green grass, a kit with a name and a position on the back, a referee who is fair and clear. Bold, fast, proud, never fussy.

Five motifs, each with a job:

1. **The pitch as the grid.** Layouts are drawn from pitch geometry: a touchline down the page edge, a halfway line between sections, the centre circle behind hero content, the penalty arc framing the primary button. Drawn as thin SVG or CSS lines in chalk white or line green, never as photos of grass.
2. **The player card.** Every athlete is shown as a portrait card: photo or silhouette, name in heavy condensed italic like the back of a shirt, position badge (GK, DF, MF, FW), LGA, and the KUID in a scoreboard-style strip. It appears on the landing page, the ID-ready screen, the profile, the club roster and the printed card. One component, many sizes.
3. **The scoreboard.** Numbers that matter (the KUID, registrations so far, the coordinator's cash total, the queue count) sit in a dark scoreboard panel with tabular digits, and count up when they first appear.
4. **The referee.** States are refereed: a whistle for information, a yellow card for something fixable, a red card for something blocked, a VAR screen for "being checked", a raised flag for "not allowed here". These are the illustrations and icons of the error system. The words stay plain and kind; the football carries the feeling.
5. **The kit colours.** Pitch green, kit red and boot black from the logo, on clean white. The logo's running figure, with arms up, is a celebration pose. Use it as the brand's motion cue: things rise and open up, like that figure.

What it is not: a betting site (no neon, no dark-gold glitz), a European club site (no stadium-photo heroes that cost 2MB), a government portal (no seals, no serif). And it must not imitate EA FC Ultimate Team cards or any league's branding. The player card is our own design.

Light first, floodlit second. The default is light, because phones are read in sunlight. Dark mode is "floodlights": deep night green with white lines. The printed card stays light in both.

Design system

All values live as CSS variables in one tokens file, exposed to Tailwind. No hex code, pixel size or duration appears anywhere else.

Colour

Taken from the logo by sampling its pixels, then measured for contrast. The logo green fails as text on white (2.99:1), so it is a fill colour, and buttons on it use black text (7.0:1), which also looks like a kit.

Token	Value	Use	Contrast measured
<code>--pitch</code>	<code>#0EAD2C</code>	Logo green. Primary buttons (black text), active tabs, success fills, pitch lines on dark	2.99:1 on white, decorative and fills only
<code>--pitch-ink</code>	<code>#087A1F</code>	Green text and links on light	5.51:1 on white
<code>--pitch-deep</code>	<code>#06661A</code>	Pressed states, headers on light	7.18:1 on white
<code>--kit-red</code>	<code>#EB0002</code>	Logo red. Live dots, highlights, card accents. Not for small text	4.63:1 on white
<code>--card-red</code>	<code>#C70003</code>	Error text, red-card state, destructive buttons	6.12:1 on white
<code>--yellow-card</code>	<code>#FFC21A</code> (to be measured)	Warning state fills, with black text	Black text on it only
<code>--boot</code>	<code>#0A0F0B</code>	Text, scoreboard panels	19.3:1 on white
<code>--chalk</code>	<code>#FFFFFF</code>	Background, lines on dark	

Token	Value	Use	Contrast measured
<code>--turf-50</code>	<code>#F6F8F3</code>	Soft section background	
<code>--night</code>	<code>#07110A</code>	Floodlit dark-mode background	Logo green on it 6.43:1

Rule: every text and background pair is measured with a contrast checker before it ships, and the numbers go in a comment beside the token. Nothing passes on eyeballing.

Type

Role	Font	Why
Display (headlines, names on cards, big numbers)	Barlow Condensed, 700 and 800, upright and italic	Closest free match to the logo's heavy condensed italic wordmark; reads like shirt lettering.
Body and UI	Atkinson Hyperlegible Next	Designed by the Braille Institute for low vision; every letter unambiguous in sunlight. A 2025 redesign, new to this project, but the same family as the old body font, so flagged here.
IDs and codes	Geist Mono	The KUID, OTP and reference codes: 0 and O, 1 and I never confused.

All three self-hosted through `next/font` as subset woff2, total under 60KB. Before choosing final weights, the agent must test that each font renders the Hausa hooked letters `ɓ` `ɗ` `ƙ` `ƴ` (and capitals). Any font that fails gets a fallback stack that has them, tested on a real Android phone.

Type scale, mobile first, fluid with `clamp()` : 14 / 16 (body, never smaller) / 18 / 22 / 28 / 36 / 48 / 64. Headlines in uppercase italic condensed for impact; body in sentence case.

Space, shape, depth

- 4px base: 4, 8, 12, 16, 24, 32, 48, 64.
- Radius: 12px on cards and sheets (a card in the hand), 999px on buttons and pills (a ball), 4px on inputs.

- Shadows: two only, a lift (cards) and a float (sheets, toasts). Depth comes mostly from layering, not blur.
- Touch targets at least 48 by 48px; primary buttons 56px tall on mobile.

Icons

One family, everywhere: **Tabler Icons** (MIT, over 5,000 icons, consistent 2px stroke, React package that renders plain inline SVG on the server, so icons work with JavaScript off). It has the football set this product needs: `ball-football`, `shirt-sport`, `soccer-field`, `play-football`, `scoreboard` ... plus `id-badge`, `qrcode`, `scan`, `trophy`, `cash`, `shield-check`, `map-pin`. Sizes 16, 20, 24, 32. Custom icons (whistle, yellow card, red card, VAR screen, position badge; Tabler has no whistle) are drawn to the same 24px grid and 2px stroke so they sit in the family.

Layout and breakpoints

Designed at 360px first, then scaled up. Tested at 320 (small Ite), 360 (most Android), 414, 768 (coordinator tablets), 1024 and 1440 (admin). Content max width 640px for athlete and public flows, 1280px for admin. Container queries for components (the player card adapts to its box, not the screen). No horizontal scroll at any width, enforced by an automated test.

Motion: smooth everywhere, required nowhere

Animation is built in three layers, so the site feels alive on a good phone and still works on Opera Mini. No screen ever waits for an animation, and no action depends on one.

Layer	Runs on	What it does	Tools
0. CSS only	Every browser, JavaScript off	Button press and hover, focus rings, field error shake, entrance fades with <code>@starting-style</code> , page-to-page morphs with cross-document View Transitions (<code>@view-transition { navigation: auto }</code>), the player card sliding from one page to the next	Plain CSS. Browsers that lack a feature skip it silently.

Layer	Runs on	What it does	Tools
1. Enhanced	JavaScript on	Step progress, the ID reveal, number count-ups, toasts, swipe on the coordinator review queue, card tilt on touch	Motion (motion.dev), loaded only inside the components that use it
2. Delight	Capable devices only: fast connection, no Save-Data, no reduced-motion, mostly desktop	Interactive hero figure and empty-state figures	Hairline custom figures, loaded when the browser is idle

Motion tokens

Token	Value	Feel
<code>--dur-tap</code>	120ms	Press feedback
<code>--dur-quick</code>	200ms	Hovers, toggles, small reveals
<code>--dur-move</code>	320ms	Sheets, toasts, step changes
<code>--dur-show</code>	560ms	Signature moments only
<code>--ease-kick</code>	cubic-bezier(.2, .8, .2, 1)	Fast out, soft landing; the default
<code>--ease-bounce</code>	cubic-bezier(.34, 1.56, .64, 1)	A small overshoot, like a ball settling; badges and success only
<code>--ease-glide</code>	cubic-bezier(.65, 0, .35, 1)	Page and layout changes

Signature moments

- Kick-off (landing).** Pitch lines draw themselves in chalk (SVG stroke animation, CSS), the headline slides in on the italic, the sample player card lifts and settles.
- The run (registration).** A progress rail along a touchline; a ball icon travels to the next marker as each part is completed. No JS: the rail simply shows where you are.
- The reveal (ID ready).** The card arrives face down, flips to the front, and an "ISSUED" stamp lands with a small bounce. Confetti in green, red and white for 1.2 seconds. This is the conversion moment, so it gets the biggest moment in the product, once.
- The net (success).** Any successful save or payment: the button's icon becomes a ball, which hits a small net that ripples. Under 600ms.

5. **The card (errors).** A yellow or red card flicks in at a slight angle and settles (see the error system).
6. **Loading.** A ball bouncing on three beats for waits under 3 seconds; pitch-line skeletons, the shape of the coming content, for longer loads.

Rules the agent must keep

- Animate only `transform` and `opacity` (and SVG stroke for line drawing). Nothing that forces layout. Target 60fps on a 2019 budget Android.
- One signature moment per screen at most.
- `prefers-reduced-motion: reduce` turns every movement into a simple fade, and confetti off.
- The `Save-Data` header or a slow connection (`navigator.connection.effectiveType` 2g or 3g) skips layer 2 entirely.
- Never animate the ID, the price or an error message's text: those must be readable from the first frame.
- Motion's JavaScript is never imported by the public profile page, which ships zero JavaScript.

Errors, empty states and waiting

Every error answers four things in this order: what happened, what is still safe, what to do (one button), and a reference code to read down the phone. The referee illustrations carry the feeling; the words stay plain, calm and never blaming.

The referee scale

Signal	Means	Colour and icon	Example
Whistle	Information, nothing wrong	Boot black on turf, whistle icon	"Your code is on its way."
Yellow card	Something to fix, you can carry on	Yellow-card fill, black text, card icon	"This phone number has 10 digits. Check it and try again."
Red card	Blocked or failed	Card-red text and icon on light red	"Payment did not go through. You were not charged."
VAR screen	Being checked, wait	Pitch-ink, TV-screen icon	"Your photo is with your LGA coordinator. Usually under 24 hours."

Signal	Means	Colour and icon	Example
Raised flag	Not allowed here	Boot black, flag icon	"This page is for coordinators. Sign in with a coordinator account."
Green tick	Done	Pitch fill, black tick	"Saved."

Five kinds of error, five treatments

1. **Field errors.** Shown under the field: red-card icon, a sentence saying how to fix it, red border plus a thicker left edge (never colour alone). The field shakes once (CSS). The typed value stays.
2. **Form summary.** At the top of the form, focused on arrival so screen readers announce it, listing each problem as a link that jumps to its field. Works with JavaScript off (plain anchor links).
3. **Page states.** A full screen with an illustration, headline, one sentence, one main button, one quiet link, and the reference chip. Needed for: not found (empty net), not allowed (raised flag), signed out (whistle: "You were signed out to keep your account safe"), too many attempts (yellow card with a countdown), KAFRIADA unreachable (red card: "We could not reach KAFRIADA. Nothing you entered was lost."), offline (signal-off icon), and server error.
4. **Flash banners.** After a form redirect, a banner slides down under the header and stays until dismissed. No-JS version arrives through the URL as today, and is never auto-dismissed.
5. **Toasts.** JavaScript only, for small confirmations ("Copied"). Never for errors that need action.

The reference chip

A monospace chip, for example `REF 7K2·Q9X`, grouped in threes so it can be read aloud. With JavaScript it copies on tap. Beside it: "Tell your LGA coordinator this code", and a WhatsApp or phone link to support once a support number exists.

Copy rules

- Say what happened in under 12 words. Then what is safe. Then the action.
- Never "invalid", "error 500", "forbidden" or a stack trace. Never blame ("you entered").
- Money errors always say whether money moved.
- Every message exists in English and Hausa in the strings file.

Empty states

Draw what is missing, not a sad face: an empty net for no search results, an empty dugout bench for no club members, a bare kit rail for no clubs yet, an empty scoreboard for no payments. Each has a sentence and the button that fills it ("Invite a player").

Waiting

Under 1 second: nothing. 1 to 3 seconds: the bouncing ball on the button. Over 3 seconds: pitch-line skeletons. Submit buttons disable and say what is happening ("Creating your ID...") so nobody taps twice; the server already rejects duplicates.

Navigation and the key screens

Each role gets its own home and its own bottom tab bar on mobile; tabs are plain links, so they work without JavaScript. The wireframe numbers (PUB-01, AUT-03...) stay the shared vocabulary in code, tests and commits.

Role	Mobile navigation	Desktop navigation	Screens
Public	Top bar: logo, language (EN / HA), Sign in. Sticky "Register free" button at the bottom of the landing page	Same top bar, wider	PUB-01 to PUB-05
Athlete	Tabs: Home · My card · Clubs · Verify · Me	Left rail with the same five	AUT-01 to 05, ATH-01 to 05, VER-01 to 05
Club admin	Athlete tabs plus a Club switcher	Rail plus club section	CLB-01 to 05
Coordinator	Tabs: Queue · Find · Pay · Cards · Cash. Cash total pinned in the top bar	Rail, two-column (queue list plus detail)	CRD-01 to 06
Admin	Not designed for phones beyond read-only	Dense sidebar, tables	ADM-01 to 07

How each launch screen should look

- **PUB-02 Landing.** Hero: "YOUR PERMANENT FOOTBALL ID. FREE." in condensed italic over a chalk-drawn centre circle, a sample player card beside it, one green button. Then a live scoreboard of registrations so far (server-rendered number), a Jigawa map with

open LGAs lit in green, three "moves" for how it works, the free-then-optional verification explained with the cash route, and a footer. Under 100KB without the optional figure.

- **AUT-01 Register.** One page with three halves: You (name, phone, date of birth), Your game (LGA, position), Your account (password, consent). The position picker is a small pitch with tappable spots for GK, DF, MF, Winger, Striker; underneath they are ordinary radio buttons, so it works without JavaScript. With JavaScript the halves reveal step by step with inline checks. Phone input shows +234 and formats as you type.
- **AUT-02 Confirm phone.** One big code field (not six boxes, which need JavaScript), "Your ID is already yours" in a green panel above the fold, resend with a visible countdown.
- **AUT-03 ID ready.** The reveal moment. The new player card, the KUID in the scoreboard strip, then three big actions: Share on WhatsApp (a `wa.me` link with the profile URL, works without JavaScript), Download card, Print. The optional ₦2,500 verification appears only below these, as an invitation, not a toll.
- **PUB-01 Public profile.** Zero JavaScript. The player card full width, "Issued by KAFRIADA" pill and "check the face" line directly under the photo, then the facts as a clean two-column list that never overflows. Verified athletes get a green shield badge; unverified get a neutral silhouette with no explanation, exactly as the wireframe says. Edge-cached.
- **ATH-03 My card / print.** Front and back at real ID-card size (85.6 by 54mm) for print, QR on the back. The backend's PNG and PDF renderer must produce the same design.
- **VER-01 to VER-05 Verification.** A three-step track (Photo, Pay, Review) shown as a match timeline. Card and cash routes side by side as equals. Under review uses the VAR state; rejected uses a yellow card and says "No second fee".
- **CRD-01 to CRD-06 Coordinator.** Built for speed in a crowded hall: today's queue count and cash total in a scoreboard header, review cards with big Approve (green) and Reject (red) buttons and the photo large, a receipt screen after every cash payment that can be shown to the athlete.
- **ADM screens.** Calm and dense: data tables with sticky headers, filters, the ledger check as the top line, the audit log with no delete control anywhere.

Slice 2 screens (SCT, TRF, FED) are designed in the same system later; transfers get a "transfer window" treatment and the feed a matchday-programme look.

Stack and performance budget

The framework stays (Next.js 15, App Router, React 19, server components, server actions), because it is what makes no-JS forms work. Everything added must either ship zero JavaScript or load only where used.

Need	Choice	Why
Styling	Tailwind CSS v4 on CSS variables	No runtime JavaScript; tokens in one place
Components	Our own small library; native <code><dialog></code> , <code><details></code> , radio and select underneath	Component kits like shadcn/Radix need JavaScript for dialogs and menus
Icons	<code>@tabler/icons-react</code>	Inline SVG, server-rendered, tree-shaken
Motion	<code>motion</code> (motion.dev) in client islands only; CSS for everything else	Smooth springs where JS exists, nothing breaks where it does not
Celebration	<code>canvas-confetti</code> (about 6KB), on the ID-ready screen only	Light, respects reduced motion
Interactive figures	<code>@lucasmakes/hairline</code> , custom figures	See the Hairline section
Translation	<code>next-intl</code> , server-side, English and Hausa	Works without client JavaScript
Forms with JS	Enhance the existing server actions; validation messages from the same strings file	One source of truth, no-JS path unchanged
Testing	Playwright screenshots at 320, 360, 768, 1440 in light and dark; the no-JS render check; axe accessibility; Lighthouse CI	Catches overflow, contrast and regressions automatically
Style guide	A <code>/styleguide</code> route in development only, showing every token and component in every state	The agent and you review the system before screens are built

Budget (fails the build if exceeded)

Page group	First load, compressed	JavaScript
PUB-01 public profile	under 60KB	none
Landing, register, sign-in	under 150KB	under 50KB, all optional
Signed-in athlete screens	under 200KB	under 90KB

Page group	First load, compressed	JavaScript
Coordinator and admin	under 300KB	as needed

Fonts under 60KB total. Images in AVIF or WebP with a tiny blurred placeholder; hero images under 40KB; no images at all when `Save-Data` is on. Largest Contentful Paint under 2.5 seconds on Lighthouse's slow-4G mobile profile.

Assets and where to get them

Everything below is free to use commercially, except the photography, where real pilot photos with signed consent beat any stock library. The logo is the one blocker: what we have is a raster PNG with black bars baked into the top and bottom, and the site needs vectors.

From you (or the logo designer)

- ☐ **Logo as SVG**, in four versions: full lockup (figure plus KAFRIADA NET plus tagline), horizontal without tagline, the running figure alone, and the wordmark alone. Plus all-white and all-black versions of each. Ask the designer for the source file (AI, SVG or Figma). If none exists, trace it in [Inkscape](#) (Path → Trace Bitmap) and clean it by hand; the agent can do a first trace.
- ☐ **Brand confirmation**: the final name and the issuing line for the card.
- ☐ **Real photos** from the Birnin Kudu pilot: players training, a registration desk, a coordinator with a tablet, a match. 15 to 20 shots, landscape and portrait, with written consent from everyone recognisable. These become the landing page and marketing.
- ☐ **Support contact** (phone or WhatsApp number) for error screens.
- ☐ **Hausa copy reviewer**.

Download

Asset	Source	License	Notes
Tabler Icons	tabler.io/icons , npm <code>@tabler/icons-react</code>	MIT	Main icon set
Barlow Condensed	Google Fonts	OFL	Display; loaded through <code>next/font</code>

Asset	Source	License	Notes
Atkinson Hyperlegible Next	Google Fonts	OFL	Body
Geist Mono	Google Fonts	OFL	IDs and codes
Motion	motion.dev , npm <code>motion</code>	MIT	Enhanced animation
canvas-confetti	GitHub , npm <code>canvas-confetti</code>	ISC	ID-ready celebration
Hairline	hairline.lucasmakes.com , npm <code>@lucasmakes/hairline</code>	MIT	Engine for custom figures
Nigeria LGA boundaries	HDX: Nigeria administrative boundaries or GRID3 Nigeria	Check each dataset's terms; HDX boundaries are usually CC BY-IGO	Filter to Jigawa's 27 LGAs, simplify to an SVG in mapshaper
Interim stock photos	Unsplash , Pexels	Unsplash / Pexels licences	Only until pilot photos exist; adult players only, since the pilot is 18+

Made by the agent (in code, no download)

- Pitch lines, centre circle, penalty arc, touchline: SVG components.
- Grass mowing stripes for dark sections: CSS gradients, zero bytes of image.
- Custom icons: whistle, yellow card, red card, VAR screen, position badges.
- Empty-state and error illustrations: empty net, raised flag, empty bench, bare kit rail, unplugged scoreboard. Simple line art in the icon style, SVG, each under 4KB.
- The player card in all sizes, and its print layout.
- Social share image (1200 by 630) generated per athlete with Next's `ImageResponse`, so a shared WhatsApp link shows the player card.

Generated from the SVG logo

File	Size	Tool
Favicon set and Apple touch icon	16, 32, 180px	RealFaviconGenerator

File	Size	Tool
App icons, including maskable	192, 512px	Maskable.app to check the safe zone
Default social image	1200 by 630	Designed in code

Tools for the job: [Squoosh](#) to compress images, [SVGOMG](#) to shrink SVGs, [WebAIM contrast checker](#) for every colour pair.

Hairline: useful, in one place

Hairline is worth using, but not as it comes. It is a library of 27 isometric line drawings (a keyboard, a vault, a padlock, a router...) that react to the pointer, MIT licensed, with no dependencies and a React wrapper that renders on the server. None of its figures is football, and most of its magic happens under a mouse, so out of the box it would feel like a tech startup, not a pitch.

What makes it valuable is that it ships its own engine and a step-by-step method (`skills/hairline-create`) for drawing new figures in the same style. The agent uses that to draw three football figures of our own:

Figure	Where	What it does
Pitch	Landing hero, desktop and fast phones	An isometric pitch of chalk lines; players (small posts) rise around the pointer or the finger, like Hairline's Terrain figure
Net	Empty search, PUB-03 no result	A goal with a bare net; the ball rolls toward it as the pointer comes near and stops short
Bench	Empty club roster (CLB-02, ATH-05)	A dugout bench with empty seats; the seat under the pointer lifts, as if waiting for a player

Rules for using it:

- Never on the public profile, the registration form or any coordinator screen. Those are for speed.
- Loaded only on idle, and only when the device passes the layer-2 check in the Motion section. Everyone else sees the same figure's still frame as a plain SVG, so the layout never jumps.
- Respects reduced motion (Hairline already does: its springs land instantly).
- Line colour from our tokens: pitch green on light, chalk white on floodlit dark.

If the three custom figures do not reach the bar in the agent's first attempt, drop them. The site does not depend on them; the CSS chalk-line drawing on the landing page does the same job more cheaply.

Build phases for the agent

Nine phases (0 to 8), launch screens first. Each phase ends with screenshots at 360px and 1440px, the no-JS render check passing, and a short note to you; the agent does not start the next phase until the current one meets its gate. Phases 0 and 1 need your five decisions from the Summary.

1. **Phase 0: Clear the ground.** Read this plan, the wireframes PDF, the backend README's eight non-negotiable rules, and the frontend-design skill in full. Create a fresh Next.js app in `kafriadadev_frontend` on a `redesign` branch, delete everything else, copy in only the kept files, write `KEPT.md`.
 - Gate: the app builds, every route returns a bare unstyled page through the kept server actions, and `KEPT.md` lists every carried-over file.
2. **Phase 1: The system.** Tokens, fonts (with the Hausa glyph test), icons, motion tokens, pitch-line SVG kit, i18n with English and Hausa files, and the `/styleguide` route.
 - Gate: you approve the style guide before any screen is built. This is the cheapest moment to change the look.
3. **Phase 2: Components.** Button, IconButton, Field, Select, PhoneInput, CodeInput, PositionPicker, Stepper, PlayerCard (all sizes plus print), KuidStrip, Scoreboard, Pill and Badge, Notice and RefereeState, RefChip, PageState, EmptyState, Skeleton, Toast, Sheet, TabBar, TopBar, DataTable. Each shown in the style guide in every state, light and dark, with JavaScript on and off.
 - Gate: axe reports no violations; every component works at 320px.
4. **Phase 3: Public and sign-up (the pilot's first number, 2,000 registrations).** PUB-01 to PUB-05, AUT-01 to AUT-05, with the kick-off, run and reveal moments.
 - Gate: a full registration completes with JavaScript off on Opera Mini (or its emulation); PUB-01 ships zero JavaScript and passes the budget; the WhatsApp share preview shows the player card.
5. **Phase 4: Athlete and verification (the second number, 5% paying).** ATH-01 to ATH-05, VER-01 to VER-05, including the cash route.
 - Gate: the Paystack sandbox payment path works end to end through the new screens; the card PNG and PDF from the backend match the new card design.
6. **Phase 5: Clubs (the third number, 25 clubs).** CLB-01 to CLB-04 with roster and bench empty state.
7. **Phase 6: Coordinator.** CRD-01 to CRD-06, tested on a 768px tablet and a 360px phone.

- Gate: approve, reject and record cash each take two taps or fewer from the queue.

8. **Phase 7: Admin.** ADM-01 to ADM-07 on desktop.

9. **Phase 8: Polish and ship.** Motion pass, Hausa review applied, PWA (if you say yes), Hairline figures, Lighthouse and accessibility audits, then retire

`kafriadadev_backend/web/` .

- Gate: every page within budget, no horizontal scroll at any tested width, every error state reachable in the style guide.

Slice 2 (scout search, transfers, feed) follows the pilot results, in the same system.

Standing rules for the agent

- Nothing from the old frontend without an entry in `KEPT.md` and a note to you.
- No business logic in the frontend; if a screen needs data the API lacks, it raises it, it does not invent it.
- No new dependency without a line in the plan's stack table.
- Every screen named by its wireframe code in commits and tests.
- If the plan and the wireframes disagree on what a screen does, the wireframes win; on how it looks, this plan wins.